

Data Engineering Project

End-to-End Batch Data Pipeline | HS26

Assessment 100% of module grade	Team project Repository + oral assessment	Milestones Weeks 3, 7 and 14
Assessment principle: The repository shows whether the pipeline works; the initial pitch and oral defences show whether students understand and can justify what they built. The midterm and final are question-only defences, with no prepared presentation or slides. No live demonstrations are required or graded.		

Part 1 — Project Description

1.1 Project overview

The goal of the project is to design, implement and explain a complete end-to-end batch data pipeline that reflects real-world data engineering practices. Each team selects a suitable dataset and develops a pipeline for a clearly defined analytics or machine-learning use case and an identified end user.

The completed pipeline must:

- ingest data from a documented source in batch mode;
- store raw and processed data using appropriate local and cloud storage;
- apply transformations that directly support the stated use case;
- serve curated data in a form usable for analysis or model development;
- use workflow orchestration for scheduling, dependencies and recovery;
- provision the required cloud infrastructure through Terraform;
- be reproducible through containerization and clear documentation; and
- be maintained in a professional GitHub repository.

1.2 Teams, ownership and individual accountability

The project is completed in teams of two students. Repository and peer-reproducibility scores are normally shared by the team. Initial-pitch and oral-defence scores may differ between team members when their demonstrated understanding or contribution differs.

- Every team member must contribute to the initial pitch and participate in the midterm and final oral defences.
- At the midterm and final defences, the instructor directs questions to both team members to assess individual understanding.
- Git history should make meaningful contributions traceable; commit count alone is not evidence of contribution.
- Students are expected to understand the complete architecture, not only the files they authored.

1.3 Use of generative AI

Generative AI tools may be used to support implementation and documentation. Students remain fully responsible for the correctness, security and maintainability of all submitted work. They must be able to explain, justify, debug and, when asked conceptually, describe how they would modify any submitted code. Inability to explain submitted work affects the oral-assessment score.

1.4 General project requirements

Area	Requirement
Use case and user	Define the problem, intended user and required data product. Transformations and models must be linked to this use case.
Data source	Document provenance, access method, format, schema characteristics, update frequency, volume and relevant data-quality risks.
Ingestion	Implement a modular batch-ingestion process and justify full versus incremental loading, ingestion frequency and failure behaviour.

Area	Requirement
Storage	Use PostgreSQL locally and appropriate cloud storage and warehouse services in the final solution.
Transformation	Clean, standardize, aggregate, engineer or remodel data as required by the use case; transformations must be justified in the project's README.
Orchestration	Schedule and coordinate pipeline steps and support reruns, retries and backfills where appropriate.
Infrastructure	Provision required Google Cloud resources with Terraform; do not hardcode credentials or secrets.
Reproducibility	A reviewer must be able to clone the repository, follow the README and reproduce the assessed pipeline stages.
Repository	Include source code, configuration, architecture diagrams, setup instructions, verification steps and known limitations.

1.5 Milestone 1 — Initial project pitch (Week 3)

The first milestone establishes a feasible project direction. Teams present:

- the selected dataset and source system;
- the user or stakeholder and the analytics or machine-learning use case;
- the expected output or data product;
- relevant data characteristics, risks and anticipated challenges;
- an initial ingestion and storage strategy; and
- Architecture v0.1, including the planned division of responsibilities.

The GitHub repository must already contain an initial README, dataset and use-case description, Architecture v0.1 and a short project plan or backlog.

Presentation format: 10 minutes per team.

1.6 Midterm deliverable (Week 7)

At the midterm, the repository must contain a reproducible local pipeline with:

- a modular batch-ingestion script that loads source data into storage;
- a local PostgreSQL database with successfully loaded and queryable data;
- a Docker Compose environment containing the required services on a common network;
- a workflow orchestrator that runs and schedules the ingestion pipeline and supports reruns or backfills;
- at least one justified transformation that supports the defined use case;
- Architecture v0.2 reflecting implementation experience and changed decisions; and
- complete instructions for setup, execution and verification.

Midterm format: a 10-minute oral defence per team, consisting entirely of instructor questions and student answers. There is no prepared presentation, slide deck or live demonstration. Both team members must answer questions about the submitted architecture, implementation, design decisions, reliability and limitations. The total includes all questions and answers. Submit the repository link and assessed commit hash or release tag through ILIAS by Thursday, 22 October 2026 (Week 6), 15:30 Europe/Zurich.

1.7 Final deliverable (Week 14)

The final repository must contain the complete end-to-end cloud pipeline, including:

- Terraform scripts that provision a Google Cloud Storage bucket and a BigQuery dataset;
- an orchestrated and schedulable pipeline from the source to the cloud data lake;
- local storage may be used for development, testing, caching or temporary processing, but the production ingestion path must not depend on data being stored locally before upload to the cloud.
- a transformation pipeline that reads from the data lake and produces curated BigQuery tables;
- An analytical data model appropriate for the defined use case. Students must clearly state what one row in each final table represents (the table's grain) and explain how the table supports analysis or model development. Where dimensional modeling is appropriate, facts and dimensions must be clearly identified.
- sensible partitioning and clustering decisions based on expected query patterns;
- data-quality checks, failure handling and safe rerun behaviour;
- complete documentation, configuration examples, verification queries and known limitations; and
- a final architecture showing the complete solution and its evolution from Architecture v0.1.

Final format: a 10-minute oral defence per team, consisting entirely of instructor questions and student answers. There is no prepared presentation, slide deck or live demonstration. Both team members must answer questions

about the completed pipeline, its results, architecture, design trade-offs, evolution and limitations. The total includes all questions and answers. Submit the repository link and assessed commit hash or release tag through ILIAS by Thursday, 10 December 2026 (Week 13), 20:00 Europe/Zurich.

Part 2 — Evaluation

2.1 Assessment model

The project constitutes 100% of the module grade. Assessment combines repository quality and reproducibility, oral assessment (the initial pitch and the midterm/final defences), and the quality of the peer reviews performed by each team.

Milestone	Instructor: repository	Peer reproducibility	Oral assessment	Review quality	Total
Initial pitch	5	–	5	–	10
Midterm	12	5	18	5	40
Final	13	5	27	5	50
Total	30	10	50	10	100

Overall weighting: 50 points assess oral understanding: 5 for the initial pitch, 18 for the midterm defence and 27 for the final defence. The remaining 50 points assess the engineering work: 30 instructor repository points, 10 peer reproducibility points and 10 points for the quality of the peer reviews performed.

2.2 Initial project pitch — 10 points

Component	Criteria	Points
Repository	Initial README; dataset and source description; use case; Architecture v0.1; initial plan	5
Initial pitch & questions	Feasibility and clarity of use case; understanding of source data; initial ingestion/storage reasoning; responses to questions	5

2.3 Midterm evaluation (Week 7) — 40 points

Component	Criteria	Points
Instructor: repository	Ingestion pipeline (4); local storage/schema (2); Docker/reproducible environment (3); orchestration and rerun behaviour (3)	12
Peer reproducibility	Independent execution of the assessed local pipeline by peer teams	5
Oral defence	The midterm oral defence assesses students' understanding of their submitted repository, including the architecture, ingestion and storage decisions, orchestration, reliability, implementation and supporting evidence. Questions are selected according to each team's repository and may explore design choices, limitations, failures or possible changes. Both students must explain and justify their work. The defence is assessed as a whole out of 18 points, without fixed sub-point allocations or a prescribed question sequence.	18
Review quality	Accuracy, completeness, evidence and usefulness of the team's peer reviews	5

2.4 Final evaluation (Week 14) — 50 points

Component	Criteria	Points
Instructor: repository	Terraform/cloud infrastructure (3); cloud ingestion (3); transformation and warehouse design (3); orchestration, reliability and data quality (2); documentation, security and reproducibility (2)	13
Peer reproducibility	Independent execution and verification of the assessed final pipeline stages by peer teams	5

Component	Criteria	Points
Oral defence	The final oral defence assesses students' understanding of their submitted end-to-end solution, including its architecture, design decisions and trade-offs, implementation, results, evolution, limitations and possible improvements. Questions are selected according to each team's repository and may ask students to explain evidence, diagnose a problem or reason about a proposed change. Both students must demonstrate their own understanding and justify their work. The defence is assessed as a whole out of 27 points, without fixed sub-point allocations or a prescribed question sequence.	27
Review quality	Accuracy, completeness, evidence and usefulness of the team's final peer reviews	5

2.5 Oral-assessment principles

- Oral assessment evaluates technical understanding and reasoning, not slide aesthetics or performance confidence. The Week 3 initial pitch remains a 10-minute presentation per team. Each midterm and final assessment is a 10-minute question-only oral defence per team; no prepared presentation or slides are required.
- No live demonstrations are required or graded at any milestone.
- During a defence, the instructor may refer to the submitted architecture diagrams, code, logs, screenshots, tests and query results. Students explain this evidence in response to questions; they do not prepare a presentation.
- Both team members must participate. The instructor directs questions to each member and distributes the available time so that both can demonstrate their understanding.
- Students must justify choices and trade-offs rather than only describe tools or steps.
- A technically polished repository cannot fully compensate for an inability to explain the submitted work.

2.6 Peer review concept

Peer review is a hands-on reproducibility and repository audit conducted after both the midterm and final submissions. It serves two purposes: it provides independent evidence about whether a repository can be reproduced, and it develops the reviewers' professional ability to validate and communicate engineering issues.

Review assignment and scoring

- Each repository is reviewed independently by two peer teams.
- Each team reviews two assigned repositories at the midterm and two at the final.
- The two reproducibility scores are averaged to produce the assessed team's peer-reproducibility score out of 5.
- The instructor may verify, moderate or override a peer score when evidence is incomplete, reviewers disagree substantially, or a failure is environment-specific.
- Separately, the instructor grades the quality of the reviews written by each team, worth 5 points at midterm and 5 points at final.

Peer-review submission

Each team must submit two peer-review documents through ILIAS, one for each assigned repository. The submission deadlines are 12 November 2026 for the midterm peer reviews and 31 December 2026 for the final peer reviews. These dates are two weeks after the corresponding defence.

Use the following filename format:

Filename: DENG_HS26_<Milestone>_Review_TeamXX_of_TeamYY.docx

Use Midterm or Final for <Milestone>. TeamXX is the reviewing team and TeamYY is the team whose repository was reviewed. Example: DENG_HS26_Midterm_Review_Team03_of_Team07.docx. Team numbers must always use two digits.

Required review procedure

1. Clone the assigned repository into a clean environment.
2. Follow the README exactly without obtaining undocumented help from the project team.
3. Attempt to configure and run the pipeline stages covered by the relevant milestone.
4. Verify the expected data, orchestration, infrastructure and outputs using the documented verification steps.
5. Record successes and failures with precise evidence including screenshots.
6. Classify failures as documentation, configuration, code, credential/cloud-permission or environment-specific issues.
7. Submit the completed rubric by the announced deadline.

Peer reproducibility score — 5 points

Score	Meaning	Evidence expectation
5	Fully reproducible	The assessed pipeline stages run from the documented instructions; only negligible clarifications are needed.
4	Mostly reproducible	The pipeline runs after minor, clearly identifiable documentation or configuration corrections.
3	Partially reproducible	Core components work, but one significant stage requires repair, undocumented intervention or missing configuration.
2	Major reproducibility problems	Several important stages fail or require substantial undocumented intervention.
1	Minimal reproducibility	Only isolated components can be verified despite a serious attempt.
0	Not reproducible	The repository cannot be meaningfully executed or essential files/instructions are absent.

Quality of the peer reviews performed — 5 points

Criterion	What is assessed	Points
Completeness	All required steps and rubric sections were attempted and documented	1
Evidence	Commands, screenshots, logs or error messages support the conclusions	1
Accuracy	Findings and scores are technically correct and consistent with the evidence	1
Diagnosis	Failures are located and classified precisely without unsupported assumptions	1
Constructive communication	Feedback is clear, professional, reproducible and actionable	1

2.7 Submission and assessment conditions

- The assessed repository state is the commit hash or release tag submitted by the deadline. Repository-link deadlines are Thursday, 22 October 2026 (Week 6), 15:30 Europe/Zurich for the midterm and Thursday, 10 December 2026 (Week 13), 20:00 Europe/Zurich for the final. Repository submissions are due one week before the corresponding midterm or final defence, at the times listed above.
- For each project milestone, the team submits one document through ILIAS containing the team number, both student names, the GitHub repository URL and the assessed commit hash or release tag.
- Private repositories must grant access to all instructors and to the peer teams assigned to review the repository. Access must be provided before the submission deadline and remain available throughout the assessment and peer-review period.
- Peer-review documents must be uploaded through ILIAS by 12 November 2026 (midterm) and 31 December 2026 (final), as specified in Section 2.6, and must follow the required filename convention.
- Required credentials must never be committed. Provide safe configuration examples such as `.env.example` and document required permissions.
- Large datasets should not be committed unless explicitly permitted; provide acquisition or generation instructions instead.
- Known limitations must be documented honestly. Documented limitations may still affect functionality scores but are preferable to hidden failures.